# TUBES I: Membuat Simulasi Antrian Perjalanan Bus



Disusun oleh:
Jethro Jens Norbert Simatupang – 13523081

**PROGRAM STUDI TEKNIK INFORMATIKA**

**SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA**

**INSTITUT TEKNOLOGI BANDUNG**

**JL. GANESA 10, BANDUNG 40132**

**2026**

## Deskripsi Masalah

Sistem yang dimodelkan adalah sistem shuttle bus yang menghubungkan dua terminal udara dengan tempat rental mobil. Sistem terdiri dari tiga lokasi, yaitu Lokasi 1 (Air Terminal 1), Lokasi 2 (Air Terminal 2), dan Lokasi 3 (Car Rental). Satu bus shuttle beroperasi dalam *loop* dengan rute berlawanan arah jarum jam, yaitu Lokasi 3 → Lokasi 1 → Lokasi 2 → Lokasi 3. Jarak perjalanan bus adalah 4,5 mil dari Lokasi 3 ke Lokasi 1, 1 mil dari Lokasi 1 ke Lokasi 2, dan 4,5 mil dari Lokasi 2 ke Lokasi 3. Bus memiliki kapasitas 20 penumpang dan bergerak dengan kecepatan 30 mil per jam.

Penumpang datang secara independen pada masing-masing lokasi dengan waktu antarkedatangan berdistribusi eksponensial. Tingkat kedatangan penumpang adalah 14 penumpang/jam di Lokasi 1, 10 penumpang/jam di Lokasi 2, dan 24 penumpang/jam di Lokasi 3. Setiap lokasi memiliki antrean FIFO dengan kapasitas tidak terbatas. Penumpang yang datang di Lokasi 1 dan Lokasi 2 memiliki tujuan ke Lokasi 3. Sementara itu, penumpang yang datang di Lokasi 3 memiliki tujuan ke Lokasi 1 dengan probabilitas 0,583 dan ke Lokasi 2 dengan probabilitas 0,417. Setelah turun di lokasi tujuannya, penumpang keluar dari sistem simulasi.

Ketika bus tiba di suatu lokasi, proses pelayanan dilakukan dengan melakukan unloading terlebih dahulu, yaitu menurunkan penumpang yang tujuan akhirnya adalah lokasi tersebut. Waktu unloading setiap penumpang berdistribusi uniform antara 16–24 detik. Setelah semua penumpang diturunkan, bus melakukan loading terhadap penumpang yang sedang menunggu di antrean selama kapasitas bus masih tersedia. Waktu loading setiap penumpang berdistribusi uniform antara 15–25 detik. Proses loading dan unloading dilakukan satu penumpang pada satu waktu.

Bus harus berada di setiap lokasi selama minimal 5 menit (minimum dwell time). Setelah proses loading dan unloading selesai serta minimum dwell time telah terpenuhi, bus meninggalkan lokasi dan pergi menuju lokasi berikutnya. Apabila proses pelayanan masih berlangsung ketika minimum dwell time tercapai, bus tetap menyelesaikan proses pelayanan terlebih dahulu sebelum berangkat.

Simulasi dilakukan menggunakan pendekatan Discrete-Event Simulation (DES) dengan bahasa pemrograman C dan library SIMLIB. Simulasi berlangsung selama 80 jam. Kejadian yang dimodelkan meliputi kedatangan penumpang, kedatangan bus, selesai unloading, selesai loading, tercapainya minimum dwell time, dan berakhirnya simulasi.

Kinerja sistem diamati melalui beberapa statistik, yaitu jumlah penumpang dalam antrean, waktu tunggu penumpang hingga mulai boarding, jumlah penumpang di dalam bus, lama bus berhenti di setiap lokasi, waktu yang diperlukan bus untuk menyelesaikan satu putaran dari Car Rental kembali ke Car Rental, serta lama waktu penumpang berada di dalam sistem hingga turun di lokasi tujuan. Statistik tersebut dihitung dalam bentuk rata-rata, maksimum, minimum, dan jumlah sampel sesuai dengan jenis statistik yang dikumpulkan.

## Source Code

*Source code* lengkap dapat diakses melalui repositori GitHub berikut:
https://github.com/JethroJNS/simlib-shuttle-bus-simulation.git

Implementasi utama program terdapat dalam berkas shuttle_bus.c dengan isi sebagai berikut:

```
#include "simlib.h"

// Tipe event
#define EVENT_ARRIVAL_1 1
#define EVENT_ARRIVAL_2 2
#define EVENT_ARRIVAL_3 3
#define EVENT_BUS_ARRIVE 4
#define EVENT_END_UNLOAD 5
#define EVENT_END_LOAD 6
#define EVENT_END_SIM 7
#define EVENT_MIN_DWELL 8

// Stream random number
#define STREAM_UNLOAD 4
#define STREAM_LOAD 5
#define STREAM_DEST 6

// 1-3 untuk queue, 4-6 untuk bus
#define LIST_QUEUE(i) (i)
#define LIST_BUS(d) (3 + (d))

// Statistik sample
#define SV_DELAY(i) (i)
#define SV_STOP(i) (3 + (i))
#define SV_LOOP 7
#define SV_SYS(i) (7 + (i))
#define TV_BUS 1

// State bus saat berhenti
#define IDLE 0
#define UNLOADING 1
#define LOADING 2

// Parameter input
double rate[4];                 // Arrival rate per jam
int capacity;                   // Kapasitas bus
double speed_mph;               // Kecepatan bus
double dist[4];                 // Jarak antar lokasi
double prob_dest[3];            // Probabilitas destinasi di LOC3
double unload_lo_s, unload_hi_s, load_lo_s, load_hi_s; // Range waktu
service
double min_dwell_min;           // Min dwell time dalam menit
double sim_hours;               // Durasi simulasi

// Parameter turunan (dalam menit)
double mean_inter[4];
double unload_lo, unload_hi, load_lo, load_hi, end_time;

// State simulasi
int num_on_bus, bus_loc, bus_state, bus_stopped, min_dwell_done;
double stop_start, last_loop_departure;

FILE *infile, *outfile;
```

```
// Deklarasi fungsi
void arrive (int loc);
void bus_arrive (void);
void serve (void);
void end_unload (void);
void end_load (void);
void min_dwell (void);
void depart (void);
void send_bus (void);
void report (void);
static void pr_samp (const char *label, int var);

int main(void) {
    int i;

    // Buka file input dan output
    infile = fopen("shuttle_bus.in", "r");
    outfile = fopen("shuttle_bus.out", "w");
    if (infile == NULL) {
        printf("Tidak bisa membuka shuttle_bus.in\n");
        return 1;
    }

    // Baca parameter dari file
    fscanf(infile, "%lf %lf %lf", &rate[1], &rate[2], &rate[3]);
    fscanf(infile, "%d %lf", &capacity, &speed_mph);
    fscanf(infile, "%lf %lf %lf", &dist[3], &dist[1], &dist[2]);
    fscanf(infile, "%lf %lf", &prob_dest[1], &prob_dest[2]);
    fscanf(infile, "%lf %lf", &unload_lo_s, &unload_hi_s);
    fscanf(infile, "%lf %lf", &load_lo_s, &load_hi_s);
    fscanf(infile, "%lf", &min_dwell_min);
    fscanf(infile, "%lf", &sim_hours);

    // Cetak parameter ke output
    fprintf(outfile, "Car-rental shuttle system\n\n");
    fprintf(outfile, "Arrival rate (per hour), location 1/2/3: %8.3f
%8.3f %8.3f\n",
            rate[1], rate[2], rate[3]);
    fprintf(outfile, "Bus capacity: %d people      Bus speed: %.2f
mph\n", capacity, speed_mph);
    fprintf(outfile, "Distances (miles): 3->1 %.2f   1->2 %.2f   2->3
%.2f\n",
            dist[3], dist[1], dist[2]);
    fprintf(outfile, "Destination probabilities for car-rental arrivals:
loc 1 %.3f   loc 2 %.3f\n",
            prob_dest[1], prob_dest[2]);
    fprintf(outfile, "Unloading time per person: U(%.1f, %.1f)
seconds\n", unload_lo_s, unload_hi_s);
    fprintf(outfile, "Loading time per person:   U(%.1f, %.1f)
seconds\n", load_lo_s, load_hi_s);
    fprintf(outfile, "Minimum stay at each location: %.2f minutes\n",
min_dwell_min);
    fprintf(outfile, "Length of simulation: %.2f hours\n", sim_hours);

    // Konversi ke menit
    for (i = 1; i <= 3; ++i)
        mean_inter[i] = 60.0 / rate[i];
    unload_lo = unload_lo_s / 60.0;
    unload_hi = unload_hi_s / 60.0;
    load_lo = load_lo_s / 60.0;
    load_hi = load_hi_s / 60.0;
```

```
    end_time = sim_hours * 60.0;

    // Inisialisasi simlib
    maxatr = 5;
    init_simlib();

    // Setup distribusi kumulatif untuk random_integer
    prob_distrib[1] = prob_dest[1];
    prob_distrib[2] = prob_dest[1] + prob_dest[2];

    // Inisialisasi state
    num_on_bus = 0;
    bus_loc = 3;
    bus_state = IDLE;
    bus_stopped = 0;
    min_dwell_done = 0;
    last_loop_departure = 0.0;
    timest(0.0, TV_BUS);

    // Jadwalkan arrival pertama
    for (i = 1; i <= 3; ++i)
        event_schedule(sim_time + expon(mean_inter[i], i), i);
    event_schedule(end_time, EVENT_END_SIM);

    // Bus berangkat dari LOC3 ke LOC1
    send_bus();

    // Main simulation loop
    do {
        timing();
        switch (next_event_type) {
            case EVENT_ARRIVAL_1:
            case EVENT_ARRIVAL_2:
            case EVENT_ARRIVAL_3: arrive(next_event_type); break;
            case EVENT_BUS_ARRIVE: bus_arrive(); break;
            case EVENT_END_UNLOAD: end_unload(); break;
            case EVENT_END_LOAD: end_load(); break;
            case EVENT_MIN_DWELL: min_dwell(); break;
            case EVENT_END_SIM: report(); break;
        }
    } while (next_event_type != EVENT_END_SIM);

    fclose(infile);
    fclose(outfile);
    return 0;
}

// Passenger arrives at a location
void arrive(int loc) {
    int dest;

    // Tentukan destinasi (LOC3 = car rental, LOC1/LOC2 = terminal)
    dest = (loc == 3) ? random_integer(prob_distrib, STREAM_DEST) : 3;

    // Simpan data passenger di queue
    transfer[1] = sim_time;
    transfer[2] = dest;
    transfer[3] = loc;
    list_file(LAST, LIST_QUEUE(loc));

    // Jadwalkan arrival berikutnya
```

```
    event_schedule(sim_time + expon(mean_inter[loc], loc), loc);

    // Jika bus sedang berhenti dan idle di lokasi ini, mulai loading
    if (bus_stopped && bus_loc == loc && bus_state == IDLE)
        serve();
}

// Bus arrives at a location
void bus_arrive(void) {
    bus_loc = (int) transfer[3];
    bus_stopped = 1;
    min_dwell_done = 0;
    stop_start = sim_time;

    // Jadwalkan event minimum dwell time
    event_schedule(sim_time + min_dwell_min, EVENT_MIN_DWELL);

    // Mulai serve (unload dulu, baru load)
    serve();
}

// Proses passenger
void serve(void) {
    int dl = LIST_BUS(bus_loc);
    double arr;
    int dest, orig;

    // Unload dulu jika ada passenger untuk lokasi ini
    if (list_size[dl] > 0) {
        list_remove(FIRST, dl);
        arr  = transfer[1];
        orig = (int) transfer[3];
        transfer[3] = arr;
        transfer[4] = orig;
        event_schedule(sim_time + uniform(unload_lo, unload_hi,
STREAM_UNLOAD), EVENT_END_UNLOAD);
        bus_state = UNLOADING;
    }
    // Load jika ada passenger di queue dan bus belum penuh
    else if (list_size[LIST_QUEUE(bus_loc)] > 0 && num_on_bus < capacity)
{
        list_remove(FIRST, LIST_QUEUE(bus_loc));
        arr  = transfer[1];
        dest = (int) transfer[2];
        orig = (int) transfer[3];
        sampst(sim_time - arr, SV_DELAY(bus_loc)); // Catat queue delay
        transfer[3] = arr;
        transfer[4] = dest;
        transfer[5] = orig;
        event_schedule(sim_time + uniform(load_lo, load_hi, STREAM_LOAD),
EVENT_END_LOAD);
        bus_state = LOADING;
    }
    // Idle (tunggu min dwell atau langsung pergi)
    else {
        bus_state = IDLE;
        if (min_dwell_done)
            depart();
    }
}
```

```
// Selesai unload satu passenger
void end_unload(void) {
    double arr = transfer[3];
    int orig = (int) transfer[4];

    num_on_bus--;
    timest((double) num_on_bus, TV_BUS);
    sampst(sim_time - arr, SV_SYS(orig)); // Catat waktu di sistem
    serve();
}

// Selesai load satu passenger
void end_load(void) {
    double arr = transfer[3];
    int dest = (int) transfer[4];
    int orig = (int) transfer[5];

    // Masukkan passenger ke list bus sesuai destinasi
    transfer[1] = arr;
    transfer[2] = dest;
    transfer[3] = orig;
    list_file(LAST, LIST_BUS(dest));
    num_on_bus++;
    timest((double) num_on_bus, TV_BUS);
    serve();
}

// Minimum dwell time selesai
void min_dwell(void) {
    min_dwell_done = 1;
    if (bus_state == IDLE)
        depart();
}

// Bus meninggalkan lokasi
void depart(void) {
    // Catat stop time
    sampst(sim_time - stop_start, SV_STOP(bus_loc));

    // Catat loop time saat departure dari LOC3
    if (bus_loc == 3) {
        sampst(sim_time - last_loop_departure, SV_LOOP);
        last_loop_departure = sim_time;
    }

    bus_stopped = 0;
    send_bus();
}

// Kirim bus ke lokasi berikutnya (counterclockwise)
void send_bus(void) {
    // Rute: 3->1, 1->2, 2->3
    int next = (bus_loc % 3) + 1;
    double travel = dist[bus_loc] / (speed_mph / 60.0); // dalam menit

    transfer[3] = next;
    event_schedule(sim_time + travel, EVENT_BUS_ARRIVE);
}

// Helper print statistik sample
static void pr_samp(const char *label, int var) {
```

```
    sampst(0.0, -var);
    fprintf(outfile, "%-26s %10.4f %10.4f %10.4f %8.0f\n", label,
            transfer[1], transfer[3], transfer[4], transfer[2]);
}

// Generate output
void report(void) {
    int i;
    char lab[64];

    fprintf(outfile, "\n\nSimulation results (times in MINUTES)\n");

    // (a) Queue length
    fprintf(outfile, "\n(a) Number in each queue\n");
    fprintf(outfile, "%-12s %12s %12s\n", "Location", "Average",
"Maximum");
    for (i = 1; i <= 3; ++i) {
        filest(LIST_QUEUE(i));
        fprintf(outfile, "%-12d %12.4f %12.0f\n", i, transfer[1],
transfer[2]);
    }

    // (b) Queue delay
    fprintf(outfile, "\n(b) Delay in each queue (arrival until boarding
begins)\n");
    fprintf(outfile, "%-26s %10s %10s %10s %8s\n", "Location", "Average",
"Maximum", "Minimum", "Count");
    for (i = 1; i <= 3; ++i) {
        sprintf(lab, "Queue %d", i);
        pr_samp(lab, SV_DELAY(i));
    }

    // (c) Bus occupancy
    fprintf(outfile, "\n(c) Number on the bus\n");
    timest(0.0, -TV_BUS);
    fprintf(outfile, "Average %10.4f   Maximum %6.0f\n", transfer[1],
transfer[2]);

    // (d) Stop time
    fprintf(outfile, "\n(d) Time bus is stopped at each location\n");
    fprintf(outfile, "%-26s %10s %10s %10s %8s\n", "Location", "Average",
"Maximum", "Minimum", "Count");
    for (i = 1; i <= 3; ++i) {
        sprintf(lab, "Location %d", i);
        pr_samp(lab, SV_STOP(i));
    }

    // (e) Loop time
    fprintf(outfile, "\n(e) Time for bus to make a loop (car rental to
car rental)\n");
    fprintf(outfile, "%-26s %10s %10s %10s %8s\n", "", "Average",
"Maximum", "Minimum", "Count");
    pr_samp("Loop time", SV_LOOP);

    // (f) Time in system
    fprintf(outfile, "\n(f) Time a person is in the system, by arrival
location\n");
    fprintf(outfile, "%-26s %10s %10s %10s %8s\n", "Arrival location",
"Average", "Maximum", "Minimum", "Count");
    for (i = 1; i <= 3; ++i) {
        sprintf(lab, "Location %d", i);
```

```
            pr_samp(lab, SV_SYS(i));
        }
    }
```

## File Input

shuttle_bus.in:

```
14 10 24
20 30
4.5 1.0 4.5
0.583 0.417
16 24
15 25
5
80
```

## File Output

shuttle_bus.out:

```
Car-rental shuttle system

Arrival rate (per hour), location 1/2/3:   14.000   10.000   24.000
Bus capacity: 20 people      Bus speed: 30.00 mph
Distances (miles): 3->1 4.50   1->2 1.00   2->3 4.50
Destination probabilities for car-rental arrivals: loc 1 0.583   loc 2
0.417
Unloading time per person: U(16.0, 24.0) seconds
Loading time per person:   U(15.0, 25.0) seconds
Minimum stay at each location: 5.00 minutes
Length of simulation: 80.00 hours


Simulation results (times in MINUTES)

(a) Number in each queue
Location          Average      Maximum
1                  7.3677           27
2                  4.8875           17
3                  9.3210           29

(b) Delay in each queue (arrival until boarding begins)
Location                      Average    Maximum    Minimum    Count
Queue 1                       31.4140    91.9342     0.0000     1120
Queue 2                       29.8196   130.8598     0.0000      784
Queue 3                       24.1302    67.4846     0.0125     1854

(c) Number on the bus
Average    13.8530   Maximum     20

(d) Time bus is stopped at each location
Location                      Average    Maximum    Minimum    Count
Location 1                     6.6982    10.1706     5.0000      110
Location 2                     5.4732     9.2866     5.0000      110
Location 3                    11.4018    13.7431     5.5718      110

(e) Time for bus to make a loop (car rental to car rental)
                              Average    Maximum    Minimum    Count
Loop time                     43.5733    48.0692    35.7146      110

(f) Time a person is in the system, by arrival location
Arrival location              Average    Maximum    Minimum    Count
Location 1                    51.9273   112.2936    17.7385     1120
Location 2                    45.5444   147.0041    13.9285      784
Location 3                    41.7239    88.2891    11.5316     1838
```